

Programação Funcional em Haskell

Estrutura dos efeitos

REALIZAR APÓS posição 35: 10.5 - Programação Funcional em Haskell: Instância de Applicative para a ADT Folds

Posições na playlist 28 a 35 | 120 minutos | 100 pontos

Curso principal: Programação Funcional em Haskell - Curso Completo - UFABC |
<https://www.youtube.com/watch?v=eTisiy5FB7k&list=PLYItvall0TqJ25sVTLcMhxsE0Hci58mpQ&index=1>

Conteúdo verificável: Functor; applicative; traversable; aplicações

Instruções

Resolva sem consultar o gabarito. Apresente desenvolvimento, hipóteses, unidades e verificações. Código pode ser escrito em papel ou editor sem sugestões automáticas; use somente recursos compatíveis com o bloco. Ferramentas de simulação são permitidas apenas onde solicitadas. Use folhas adicionais para diagramas e cálculos.

Pontuação: Q1 = 15; Q2 = 25; Q3 = 25; Q4 = 35. Crédito parcial depende de raciocínio demonstrado. Meta: ≥ 80 prosseguir; 70–79 revisar erros; < 70 revisar o bloco.

Aluno(a): _____ Data: _____

Alinhamento: títulos/ordem e informações públicas do curso. As aulas não foram integralmente assistidas; detalhes não expostos não estão certificados.

Functor

Para data Caixa $a = \text{Caixa } a$: (a) defina fmap. (b) Verifique leis de identidade e composição. (c) Explique por que mudar o valor preserva estrutura, mas não implica execução de IO.

Desenvolvimento / justificativa / esquema:

Applicative

Use Maybe: (a) avalie $(+) \langle \$ \rangle \text{Just2} \langle * \rangle \text{Just3}$ e $(+) \langle \$ \rangle \text{Nothing} \langle * \rangle \text{Just3}$. (b) Dê definição conceitual de pure e $\langle * \rangle$. (c) Compare independência estrutural dos contextos com encadeamento que depende de um valor anterior.

Desenvolvimento / justificativa / esquema:

Traversable

(a) Avalie `sequenceA[Just1, Just2]` e `sequenceA[Just1, Nothing]`. (b) Use `traverse` para validar lista de inteiros não negativos, retornando `Nothing` se algum falhar. (c) Explique diferença para `map`, que deixaria contextos separados.

Desenvolvimento / justificativa / esquema:

Leis de Functor e estrutura

Considere data $\text{Par } a = \text{Par } a \ a$. Uma proposta é $\text{fmapRuim } f (\text{Par } x \ y) = \text{Par } (f \ y) (f \ x)$. (a) Teste a lei de identidade em $\text{Par } 1 \ 2$. (b) Escreva fmap correto e prove identidade e composição. (c) Explique como uma instância Applicative que aplica funções por posição pode preservar a estrutura de Par .

Desenvolvimento / justificativa / esquema:

Gabarito e Resoluções

Abra esta seção somente depois de concluir a tentativa. Há mais de um método correto possível; compare também o processo e os casos extremos.

Critérios de correção

Q1: 4 pontos pela modelagem/conceito, 8 pela solução, 3 pela verificação. Q2 e Q3: 6 pela modelagem, 14 pela solução, 5 pela justificativa/verificação. Q4: 8 pela modelagem, 20 pela solução, 7 pela análise crítica e casos extremos. Nos itens dissertativos, a solução inclui os conceitos e relações obrigatórios; crédito integral exige conectar esses conceitos ao cenário. Em código, valorize correção do contrato, casos base e explicação; uma chamada a biblioteca que evita a implementação pedida não substitui o algoritmo.

Questão 1 - Functor (15 pontos)

$fmap\ f\ (Caixa\ x) = Caixa(f\ x)$. Identidade: $fmap\ id\ (Caixa\ x) = Caixa\ x$. Composição: $fmap\ (g.f)\ (Caixa\ x) = Caixa(g(f\ x)) = fmap\ g(fmap\ f\ (Caixa\ x))$. A estrutura externa `Caixa` permanece. Functor não é sinônimo de efeito `doIO`; é abstração de mapeamento em contexto. Cobrir todas as alternativas em tipos com mais construtores.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 2 - Applicative (25 pontos)

`ResultadosJust5` e `Nothing`. `pure` injeta valor no contexto; `(<*>)` aplica função contextual a argumento contextual. A forma `applicative` combina estrutura de efeitos determinada pelas partes; um passo cuja próxima ação depende do valor extraído pode precisar de encadeamento monádico. Não confundir `Nothing` com `número0`, nem aplicar soma diretamente a `Maybe` sem desembrulhar/combinar contexto.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 3 - Traversable (25 pontos)

`sequenceA` produz `Just[1, 2]` e `Nothing`. Validador `v x = if x >= 0 then Just x else Nothing`; `traverse v xs` combina validações em `Maybe [Int]`. `map v xs` produz lista de `Maybe`, sem reunir falha/sucesso global. Travessia preserva forma da estrutura original dentro do contexto, combinando segundo a instância `applicative`.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 4 - Leis de Functor e estrutura (35 pontos)

`fmapRuim id (Par 1 2) = Par 2 1`, diferente de `Par 1 2`: viola identidade. Uma implementação correta é:

```
instance Functor Par where
  fmap f (Par x y) = Par (f x) (f y)
```

Identidade mantém ambos elementos; composição aplica $g(f(x))$ e $g(f(y))$, igual a dois mapeamentos sucessivos. Applicative por posição pode definir $\text{pure } x = \text{Par } x \text{ x}$ e $\text{Par } f \text{ g } \langle * \rangle \text{ Par } x \text{ y} = \text{Par } (f \text{ x}) (g \text{ y})$. O formato de dois campos permanece; essa instância não combina todas as posições como a instância usual de listas. Crédito exige evidenciar a lei no exemplo e justificar o mapeamento correto.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Diagnóstico do checkpoint

Registre sua nota e os conceitos a revisar em progresso.csv. Antes de avançar, refaça os itens errados sem olhar a resolução e crie um pequeno exemplo novo que use a mesma ideia. Nota alta sem justificar respostas não demonstra domínio.